

Объявление

```
T* operator+=(ptrdiff_t i) volatile noexcept;
T* operator+=(ptrdiff_t i) noexcept;
```

Результат

```
return this->fetch_add(i) + i;
```

Составной оператор присваивания `std::atomic<T*>::operator--`

Атомарно вычитает предоставленное значение из значения, сохраненного в `*this`, применяя стандартные правила арифметики указателей, и возвращает новое значение.

Объявление

```
T* operator--(ptrdiff_t i) volatile noexcept;
T* operator--(ptrdiff_t i) noexcept;
```

Результат

```
return this->fetch_sub(i) - i;
```

Г.4. Заголовок <future>

Заголовок <future> предоставляет средства обработки результатов асинхронных операций, которые могли быть выполнены в другом потоке.

Содержимое заголовка

```
namespace std
{
    enum class future_status {
        ready, timeout, deferred };
    enum class future_errc
    {
        broken_promise,
        future_already_retrieved,
        promise_already_satisfied,
        no_state
    };

    class future_error;

    const error_category& future_category();
    error_code make_error_code(future_errc e);
    error_condition make_error_condition(future_errc e);

    template<typename ResultType>
    class future;

    template<typename ResultType>
    class shared_future;

    template<typename ResultType>
    class promise;
```

```

template<typename FunctionSignature>
class packaged_task; // определение не предоставляется

template<typename ResultType, typename ... Args>
class packaged_task<ResultType (Args...)>;

enum class launch {
    async, deferred
};

template<typename FunctionType, typename ... Args>
future<result_of<FunctionType(Args...)>::type>
async(FunctionType&& func, Args&& ... args);

template<typename FunctionType, typename ... Args>
future<result_of<FunctionType(Args...)>::type>
async(std::launch policy, FunctionType&& func, Args&& ... args);
}

```

Г.4.1. Шаблон класса `std::future`

Шаблон класса `std::future` предоставляет средства для ожидания результата асинхронной операции, выполненной в другом потоке, в сочетании с шаблонами классов `std::promise` и `std::packaged_task` и шаблоном функции `std::async`, которые могут использоваться для предоставления результата выполнения асинхронной операции. На конкретный результат выполнения асинхронной операции в любой момент может ссылаться только один экземпляр `std::future`.

Экземпляры `std::future` могут быть `MoveConstructible` и `MoveAssignable`, но не `CopyConstructible` или `CopyAssignable`.

Определение класса

```

template<typename ResultType>
class future
{
public:
    future() noexcept;
    future(future&&) noexcept;
    future& operator=(future&&) noexcept;
    ~future();

    future(future const&) = delete;
    future& operator=(future const&) = delete;

    shared_future<ResultType> share();

    bool valid() const noexcept;

    см. описание get();

    void wait();

```

```
template<typename Rep,typename Period>
future_status wait_for(
    std::chrono::duration<Rep,Period> const& relative_time);

template<typename Clock,typename Duration>
future_status wait_until(
    std::chrono::time_point<Clock,Duration> const& absolute_time);
};
```

Конструктор по умолчанию std::future

Создает объект `std::future` без связанного с ним результата выполнения асинхронной операции.

Объявление

```
future() noexcept;
```

Результат

Создает новый экземпляр `std::future`.

Постусловия

`valid()` возвращает `false`.

Выдает

Ничего не выдает.

Перемещающий конструктор std::future

Создает один объект `std::future` из другого, передавая принадлежность результата выполнения асинхронной операции, связанного с другим объектом `std::future`, в только что созданный экземпляр.

Объявление

```
future(future&& other) noexcept;
```

Результат

Создает новый экземпляр `std::future`, перемещая в него содержимое объекта `other`.

Постусловия

Результат выполнения асинхронной операции, связанный с объектом `other` до вызова конструктора, связывается с только что созданным объектом `std::future`. У объекта `other` больше нет связанного с ним результата выполнения асинхронной операции. Выражение `this->valid()` возвращает тот же результат, который возвращался выражением `other.valid()` до вызова этого конструктора. Выражение `other.valid()` возвращает `false`.

Выдает

Ничего не выдает.

Перемещающий оператор присваивания `std::future`

Передаёт принадлежность результата выполнения асинхронной операции, связанного с одним объектом `std::future`, другому объекту.

Объявление

```
future(future&& other) noexcept;
```

Результат

Передаёт принадлежность асинхронного состояния между экземплярами `std::future`.

Постусловия

Результат выполнения асинхронной операции, связанный с объектом `other` до вызова конструктора, связывается с `*this`, а у `other` связанного с ним результата выполнения асинхронной операции больше не остается. Принадлежность асинхронного состояния (если таковое имеется), связанного с `*this` до совершения вызова, и состояние, если это последняя ссылка, уничтожаются. Выражение `this->valid()` возвращает тот же результат, который возвращает выражением `other.valid()` до вызова этого конструктора. Выражение `other.valid()` возвращает `false`.

Выдает

Ничего не выдает.

Деструктор `std::future`

Уничтожает объект `std::future`.

Объявление

```
~future();
```

Результат

Уничтожает `*this`. Если это последняя ссылка на результат выполнения асинхронной операции, связанная с `*this` (если таковая имеется), то уничтожается и этот результат выполнения асинхронной операции.

Выдает

Ничего не выдает.

Компонентная функция `std::future::share`

Создает новый экземпляр `std::shared_future` и передает принадлежность результата выполнения асинхронной операции, связанной с `*this`, только что созданному экземпляру `std::shared_future`.

Объявление

```
shared_future<ResultType> share();
```

Результат

Как при вычислении `shared_future<ResultType>(std::move(*this))`.

Постусловия

Результат выполнения асинхронной операции, связанный с `*this` до вызова `share()` (если такой имелся), связывается с только что созданным экземпляром `std::shared_future`. Выражение `this->valid()` возвращает `false`.

Выдает

Ничего не выдает.

Компонентная функция `std::future::valid`

Проверяет, связан ли экземпляр `std::future` с результатом выполнения асинхронной операции.

Объявление

```
bool valid() const noexcept;
```

Возвращает

`true`, если у `*this` есть связанный с ним результат выполнения асинхронной операции, или `false` в противном случае.

Выдает

Ничего не выдает.

Компонентная функция `std::future::wait`

Вызывает отложенную функцию, если она содержится в состоянии, связанном с `*this`, в противном случае ожидает готовности результата выполнения асинхронной операции, связанной с экземпляром `std::future`.

Объявление

```
void wait();
```

Предусловия

Выражение `this->valid()` должно возвращать `true`.

Результат

Если связанное состояние содержит отложенную функцию, вызывает эту функцию и сохраняет возвращенное значение или выданное исключение в качестве результата выполнения асинхронной операции. В противном случае блокируется, пока не будет готов результат выполнения асинхронной операции, связанной с `*this`.

Выдает

Ничего не выдает.

Компонентная функция `std::future::wait_for`

Ожидает, пока не будет готов результат выполнения асинхронной операции, связанной с экземпляром `std::future`, или пока не истечет заданный период времени.

Объявление

```
template<typename Rep,typename Period>
future_status wait_for(
    std::chrono::duration<Rep,Period> const& relative_time);
```

Предусловия

Выражение `this->valid()` должно возвращать `true`.

Результат

Если результат выполнения асинхронной операции, связанной с `*this`, содержит отложенную функцию, актуализируемую из вызова `std::async`, выполнение которой еще не началось, происходит немедленный возврат управления без блокировки. В противном случае происходит блокировка до готовности результата выполнения асинхронной операции, связанной с `*this`, или до истечения периода времени, заданного аргументом `relative_time`.

Возвращает

`std::future_status::deferred`, если результат выполнения асинхронной операции, связанной с `*this`, содержит отложенную функцию, актуализируемую из вызова `std::async`, выполнение которой еще не началось, `std::future_status::ready`, если готов результат выполнения асинхронной операции, связанной с `*this`, `std::future_status::timeout`, если истек период времени, заданный аргументом `relative_time`.

ПРИМЕЧАНИЕ

Поток может быть заблокирован на время дольше указанного. По возможности истекшее время нужно измерять по стабильным часам.

Выдает

Ничего не выдает.

Компонентная функция `std::future::wait_until`

Ожидает, пока не будет готов результат выполнения асинхронной операции, связанной с экземпляром `std::future`, или пока не истечет заданный период времени.

Объявление

```
template<typename Clock,typename Duration>
future_status wait_until(
    std::chrono::time_point<Clock,Duration> const& absolute_time);
```

Предусловия

Выражение `this->valid()` должно возвращать `true`.

Результат

Если результат выполнения асинхронной операции, связанной с `*this`, содержит отложенную функцию, актуализируемую из вызова `std::async`, выполнение которой

еще не началось, происходит немедленный возврат управления без блокировки. В противном случае происходит блокировка до готовности результата выполнения асинхронной операции, связанной с `*this`, или до тех пор, пока вызов функции `Clock::now()` не вернет время, равное времени, заданному аргументом `absolute_time`, или более позднее.

Возвращает

`std::future_status::deferred`, если результат выполнения асинхронной операции, связанной с `*this`, содержит отложенную функцию, актуализируемую из вызова `std::async`, выполнение которой еще не началось, `std::future_status::ready`, если готов результат выполнения асинхронной операции, связанной с `*this`, `std::future_status::timeout`, если в результате вызова функции `Clock::now()` возвращено время, равное времени, заданному аргументом `absolute_time`, или более позднее.

ПРИМЕЧАНИЕ

По продолжительности блокировки вызывающего потока не дается никаких гарантий, если же функция возвращает `std::future_status::timeout`, то гарантируется только то, что функция `Clock::now()` возвращает время, равное времени `absolute_time` на тот момент, когда поток становится разблокированным, или более позднее.

Выдает

Ничего не выдает.

Компонентная функция `std::future::get`

Если связанное состояние содержит отложенную функцию из вызова `std::async`, вызывает эту функцию и возвращает результат. В противном случае ожидает готовность результата выполнения асинхронной операции, связанной с экземпляром `std::future`, а затем возвращает сохраненное значение или выдает сохраненное исключение.

Объявление

```
void future<void>::get();  
R& future<R&>::get();  
R future<R>::get();
```

Предусловия

Выражение `this->valid()` должно возвращать `true`.

Результат

Если состояние, связанное с `*this`, содержит отложенную функцию, происходит вызов этой функции и возвращается результат или же выполняется распространение любого выданного исключения. В противном случае происходит блокировка, пока не будет готов результат выполнения асинхронной операции, связанной с `*this`. Если результатом будет сохраненное исключение, происходит выдача этого исключения. В противном случае возвращается сохраненное значение.

Возвращает

Если связанное состояние содержит отложенную функцию, возвращается результат вызова этой функции. В противном случае, если тип `ResultType` относится к `void`, возвращается просто управление. Если типом `ResultType` является `R&` для некоторого типа `R`, возвращается сохраненная ссылка. В противном случае возвращается сохраненное значение.

Выдает

Исключение, выданное путем отложенного исключения или сохраненное в качестве результата выполнения асинхронной операции, если таковое имеется.

Постусловие

```
this->valid() == false
```

Г.4.2. Шаблон класса `std::shared_future`

Шаблон класса `std::shared_future` предоставляет средства ожидания результата выполнения асинхронной операции от другого потока совместно с шаблонами классов `std::promise` и `std::packaged_task` и шаблоном функции `std::async`, который может применяться для предоставления этого результата.

Экземпляры `std::shared_future` являются `CopyConstructible` и `CopyAssignable`. Объект `std::shared_future` можно также создать за счет перемещения из объекта `std::future` с тем же `ResultType`.

Обращения к конкретному экземпляру `std::shared_future` не синхронизированы. Поэтому обращения к одному и тому же экземпляру `std::shared_future` из нескольких потоков без внешней синхронизации небезопасны. Но обращения к связанному состоянию синхронизированы, поэтому безопасно обращаться без внешней синхронизации к разным экземплярам `std::shared_future`, совместно использующим связанное состояние, могут сразу несколько потоков.

Определение класса

```
template<typename ResultType>
class shared_future
{
public:
    shared_future() noexcept;
    shared_future(future<ResultType>&&) noexcept;
    shared_future(shared_future&&) noexcept;
    shared_future(shared_future const&);
    shared_future& operator=(shared_future const&);
    shared_future& operator=(shared_future&&) noexcept;
    ~shared_future();

    bool valid() const noexcept;

    см. описание get() const;

    void wait() const;

    template<typename Rep, typename Period>
```

```
future_status wait_for(  
    std::chrono::duration<Rep,Period> const& relative_time) const;  
template<typename Clock,typename Duration>  
future_status wait_until(  
    std::chrono::time_point<Clock,Duration> const& absolute_time) const;  
};
```

Конструктор по умолчанию std::shared_future

Создает объект `std::shared_future` без связанного результата выполнения асинхронной операции.

Объявление

```
shared_future() noexcept;
```

Результат

Создает новый экземпляр `std::shared_future`.

Постусловия

Для только что созданного экземпляра функция `valid()` возвращает `false`.

Выдает

Ничего не выдает.

Перемещающий конструктор std::shared_future

Создает один объект `std::shared_future` из другого, перемещая принадлежность результата выполнения асинхронной операции, связанного с другим объектом `std::shared_future`, в только что созданный экземпляр.

Объявление

```
shared_future(shared_future&& other) noexcept;
```

Результат

Создает новый экземпляр `std::shared_future`.

Постусловия

Результат выполнения асинхронной операции, связанный с объектом `other` до вызова конструктора, связывается с только что созданным объектом `std::shared_future`. У объекта `other` связанного с ним результата выполнения асинхронной операции больше нет.

Выдает

Ничего не выдает.

Конструктор перемещения в std::shared_future из std::future

Создает объект `std::shared_future` из объекта `std::future`, перемещая принадлежность связанного с `std::future` результата выполнения асинхронной операции в только что созданный объект `std::shared_future`.

Объявление

```
shared_future(std::future<ResultType>&& other) noexcept;
```

Результат

Создает новый экземпляр `std::shared_future`.

Постусловия

Результат выполнения асинхронной операции, связанный с другим объектом до вызова конструктора, связывается с только что созданным объектом `std::shared_future`. У другого объекта больше нет связанного с ним результата выполнения асинхронной операции.

Выдает

Ничего не выдает.

Копирующий конструктор `std::shared_future`

Создает один объект `std::shared_future` из другого таким образом, что и исходный объект и его копия ссылаются на результат выполнения асинхронной операции, связанный с исходным объектом `std::shared_future`, если таковой результат имелся.

Объявление

```
shared_future(shared_future const& other);
```

Результат

Создает новый экземпляр `std::shared_future`.

Постусловия

Результат выполнения асинхронной операции, связанный с другим объектом до вызова конструктора, связывается с только что созданным объектом `std::shared_future` и с другим объектом.

Выдает

Ничего не выдает.

Деструктор `std::shared_future`

Уничтожает объект `std::shared_future`.

Объявление

```
~shared_future();
```

Результат

Уничтожает `*this`. Если больше нет экземпляра `std::promise` или экземпляра `std::packaged_task`, связанного с результатом выполнения асинхронной операции, который также связан с `*this`, и это последний экземпляр `std::shared_future`, связанный с результатом выполнения асинхронной операции, то уничтожается и этот результат.

Выдает

Ничего не выдает.

Компонентная функция `std::shared_future::valid`

Проверяет, связан ли экземпляр `std::shared_future` с результатом выполнения асинхронной операции.

Объявление

```
bool valid() const noexcept;
```

Возвращает

`true`, если у `*this` есть связанный с ним результат выполнения асинхронной операции, или `false` в противном случае.

Выдает

Ничего не выдает.

Компонентная функция `std::shared_future::wait`

Если состояние, связанное с `*this`, содержит отложенную функцию, вызывает эту функцию. В противном случае ожидает готовности результата выполнения асинхронной операции, связанной с экземпляром `std::shared_future`.

Объявление

```
void wait() const;
```

Предусловия

Выражение `this->valid()` должно возвращать `true`.

Результат

Вызовы функций `get()` и `wait()` из нескольких потоков в отношении экземпляров `std::shared_future`, совместно использующих одно и то же связанное состояние, выполняются последовательно. Если связанное состояние содержит отложенную функцию, то первый вызов `get()` или `wait()` приводит к вызову отложенной функции и результатом вызова асинхронной операции становится сохранение возвращенного значения или выдача исключения. Ставит блокировку до готовности результата выполнения асинхронной операции, связанной с `*this`.

Выдает

Ничего не выдает.

Компонентная функция `std::shared_future::wait_for`

Ожидает, пока не будет готов результат выполнения асинхронной операции, связанной с экземпляром `std::shared_future`, или пока не истечет заданный период времени.

Объявление

```
template<typename Rep,typename Period>
future_status wait_for(
    std::chrono::duration<Rep,Period> const& relative_time) const;
```

Предусловия

Выражение `this->valid()` должно возвращать `true`.

Результат

Если результат выполнения асинхронной операции, связанной с `*this`, содержит отложенную функцию, актуализируемую из вызова `std::async`, выполнение которой еще не началось, происходит немедленный возврат управления без блокировки. В противном случае происходит блокировка до готовности результата выполнения асинхронной операции, связанной с `*this`, или до истечения периода времени, заданного аргументом `relative_time`.

Возвращает

`std::future_status::deferred`, если результат выполнения асинхронной операции, связанной с `*this`, содержит отложенную функцию, актуализируемую из вызова `std::async`, выполнение которой еще не началось, `std::future_status::ready`, если готов результат выполнения асинхронной операции, связанной с `*this`, `std::future_status::timeout`, если истек период времени, заданный аргументом `relative_time`.

ПРИМЕЧАНИЕ

Поток может быть заблокирован на время дольше указанного. По возможности истекшее время нужно измерять по стабильным часам.

Выдает

Ничего не выдает.

Компонентная функция `std::shared_future::wait_until`

Ожидает, пока не будет готов результат выполнения асинхронной операции, связанной с экземпляром `std::shared_future`, или пока не истечет заданный период времени.

Объявление

```
template<typename Clock,typename Duration>
bool wait_until(
    std::chrono::time_point<Clock,Duration> const& absolute_time) const;
```

Предусловия

Выражение `this->valid()` должно возвращать `true`.

Результат

Если результат выполнения асинхронной операции, связанной с `*this`, содержит отложенную функцию, актуализируемую из вызова `std::async`, выполнение которой

еще не началось, происходит немедленный возврат управления без блокировки. В противном случае происходит блокировка до готовности результата выполнения асинхронной операции, связанной с `*this`, или до тех пор, пока вызов функции `Clock::now()` не вернет время, равное времени, заданному аргументом `absolute_time`, или более позднее.

Возвращает

`std::future_status::deferred`, если результат выполнения асинхронной операции, связанной с `*this`, содержит отложенную функцию, актуализируемую из вызова `std::async`, выполнение которой еще не началось, `std::future_status::ready`, если готов результат выполнения асинхронной операции, связанной с `*this`, `std::future_status::timeout`, если в результате вызова функции `Clock::now()` возвращено время, равное времени, заданному аргументом `absolute_time`, или более позднее.

ПРИМЕЧАНИЕ

По продолжительности блокировки вызывающего потока не дается никаких гарантий, если же функция возвращает `std::future_status::timeout`, то гарантируется только то, что функция `Clock::now()` возвращает время, равное времени `absolute_time` на тот момент, когда поток становится разблокированным, или более позднее.

Выдает

Ничего не выдает.

Компонентная функция `std::shared_future::get`

Если связанное состояние содержит отложенную функцию из вызова `std::async`, вызывает эту функцию и возвращает результат. В противном случае ожидает готовность результата выполнения асинхронной операции, связанной с экземпляром `std::future`, а затем возвращает сохраненное значение или выдает сохраненное исключение.

Объявление

```
void shared_future<void>::get() const;  
R& shared_future<R&>::get() const;  
R const& shared_future<R>::get() const;
```

Предусловия

Выражение `this->valid()` должно возвращать `true`.

Результат

Вызовы функций `get()` и `wait()` из нескольких потоков в отношении экземпляров `std::shared_future`, совместно использующих одно и то же связанное состояние, выполняются последовательно. Если связанное состояние содержит отложенную функцию, то первый вызов `get()` или `wait()` приводит к вызову отложенной функции, и результатом вызова асинхронной операции становится сохранение

возвращенного значения или выданного исключения. Ставит блокировку до готовности результата выполнения асинхронной операции, связанной с **this*. Если в результате выполнения асинхронной операции хранится исключение, выдает это исключение. В противном случае возвращает сохраненное значение.

Возвращает

Если тип `ResultType` относится к `void`, возвращается просто управление. Если типом `ResultType` является `R&` для некоторого типа `R`, возвращается сохраненная ссылка. В противном случае возвращается `const`-ссылка на сохраненное значение.

Выдает

Сохраненное исключение, если таковое имеется.

Г.4.3. Шаблон класса `std::packaged_task`

Шаблон класса `std::packaged_task` упаковывает функцию или другой вызываемый объект, поэтому при вызове функции через экземпляр `std::packaged_task` результат сохраняется в качестве результата выполнения асинхронной операции, который можно извлечь через экземпляр `std::future`.

Экземпляры `std::packaged_task` могут быть `MoveConstructible` и `MoveAssignable`, но не `CopyConstructible` и не `CopyAssignable`.

Определение класса

```
template<typename FunctionType>
class packaged_task; // не определен

template<typename ResultType, typename... ArgTypes>
class packaged_task<ResultType(ArgTypes...)>
{
public:
    packaged_task() noexcept;
    packaged_task(packaged_task&&) noexcept;
    ~packaged_task();

    packaged_task& operator=(packaged_task&&) noexcept;
    packaged_task(packaged_task const&) = delete;
    packaged_task& operator=(packaged_task const&) = delete;

    void swap(packaged_task&) noexcept;

    template<typename Callable>
    explicit packaged_task(Callable&& func);

    template<typename Callable, typename Allocator>
    packaged_task(std::allocator_arg_t, const Allocator&, Callable&&);

    bool valid() const noexcept;
    std::future<ResultType> get_future();
    void operator()(ArgTypes...);
    void make_ready_at_thread_exit(ArgTypes...);
    void reset();
};
```

Конструктор по умолчанию `std::packaged_task`

Создает объект `std::packaged_task`.

Объявление

```
packaged_task() noexcept;
```

Результат

Создает экземпляр `std::packaged_task` без связанной задачи или связанного результата выполнения асинхронной операции.

Выдает

Ничего не выдает.

Конструирование `std::packaged_task` из вызываемого объекта

Создает объект `std::packaged_task` со связанной задачей и связанным результатом выполнения асинхронной операции.

Объявление

```
template<typename Callable>  
packaged_task(Callable&& func);
```

Предусловия

Выражение `func(args...)`, в котором каждый элемент `args-i` в списке `args...` должен быть значением соответствующего типа `ArgTypes-i` из списка `ArgTypes...`, должно быть допустимым. Возвращаемое значение должно допускать преобразование в `ResultType`.

Результат

Создает экземпляр `std::packaged_task` со связанным, но еще не готовым результатом выполнения асинхронной операции типа `ResultType` и со связанной задачей типа `Callable`, являющейся копией `func`.

Выдает

Исключение типа `std::bad_alloc`, если конструктор не в состоянии распределить память для результата выполнения асинхронной операции. Любое исключение, выданное в результате вызова конструктора копирования или конструктора перемещения объекта `Callable`.

Конструирование `std::packaged_task` из вызываемого объекта с распределителем

Создает объект `std::packaged_task` со связанной задачей и связанным результатом выполнения асинхронной операции, используя предоставленный распределитель памяти под результат выполнения асинхронной операции и задачу.

Объявление

```
template<typename Allocator, typename Callable>  
packaged_task(  
    std::allocator_arg_t, Allocator const& alloc, Callable&& func);
```

Предусловия

Выражение `func(args...)`, в котором каждый элемент `args-i` в списке `args...` должен быть значением соответствующего типа `ArgTypes-i` из списка `ArgTypes...`, должно быть допустимым. Возвращаемое значение должно допускать преобразование в `ResultType`.

Результат

Создает экземпляр `std::packaged_task` со связанным, но еще не готовым результатом выполнения асинхронной операции типа `ResultType` и связанной задачей типа `Callable`, являющейся коннечей `func`. Память для результата выполнения асинхронной операции и операции распределяется посредством распределителя `alloc` или через соответствующую коннечу.

Выдает

Любое исключение, выданное распределителем при попытке распределения памяти под результат выполнения асинхронной операции или под задачу. Любое исключение, выданное в результате вызова конструктора копирования или конструктора перемещения объекта `Callable`.

Перемещающий конструктор `std::packaged_task`

Создает один объект `std::packaged_task` из другого, перемещая принадлежность связанных с другим объектом `std::packaged_task` результата выполнения асинхронной операции и задачи в только что созданный экземпляр.

Объявление

```
packaged_task(packaged_task&& other) noexcept;
```

Результат

Создает новый экземпляр `std::packaged_task`.

Постусловия

Связанные до вызова конструктора с `other` результат выполнения асинхронной операции и задача становятся связанными с только что созданным объектом `std::packaged_task`. У объекта `other` связанного с ним результата выполнения асинхронной операции больше нет.

Выдает

Ничего не выдает.

Перемещающий оператор присваивания `std::packaged_task`

Перемещает принадлежность результата выполнения асинхронной операции, связанного с одним объектом `std::packaged_task` в другой объект.

Объявление

```
packaged_task& operator=(packaged_task&& other) noexcept;
```

Результат

Перемещает принадлежность результата выполнения асинхронной операции и задачи, связанной с `other`, в `*this` и сбрасывает любой прежний результат выполнения асинхронной операции, как при вычислении `std::packaged_task(other).swap(*this)`.

Постусловия

Связанные до вызова перемещающего оператора присваивания с `other` результат выполнения асинхронной операции и задача становятся связанными с `*this`. У объекта `other` связанного с ним результата выполнения асинхронной операции больше нет.

Возвращает

`*this`

Выдает

Ничего не выдает.

Компонентная функция `std::packaged_task::swap`

Производит обмен принадлежности результатов выполнения асинхронной операции между двумя объектами `std::packaged_task`.

Объявление

```
void swap(packaged_task& other) noexcept;
```

Результат

Производит обмен принадлежности результатов выполнения асинхронной операции и задач, связанных с `other` и `*this`.

Постусловия

Результат выполнения асинхронной операции и задача, связанные с `other` до вызова `swap` (если таковые имелись), связаны с `*this`. Результат выполнения асинхронной операции и задача, связанные с `*this` до вызова `swap` (если таковые имелись), связаны с `other`.

Выдает

Ничего не выдает.

Деструктор `std::packaged_task`

Уничтожает объект `std::packaged_task`.

Объявление

```
~packaged_task();
```

Результат

Уничтожает `*this`. Если у `*this` есть связанный с ним результат выполнения асинхронной операции и этот результат не содержит сохраненной задачи или

исключения, он становится готовым с исключением `std::future_error`, содержащим код ошибки `std::future_errc::broken_promise`.

Выдает

Ничего не выдает.

Компонентная функция `std::packaged_task::get_future`

Извлекает экземпляр `std::future` для результата выполнения асинхронной операции, связанного с `*this`.

Объявление

```
std::future<ResultType> get_future();
```

Предусловия

У `*this` есть связанный с ним результат выполнения асинхронной операции.

Возвращает

Экземпляр `std::future` для результата выполнения асинхронной операции, связанного с `*this`.

Выдает

Исключение типа `std::future_error` с кодом ошибки `std::future_errc::future_already_retrieved`, если объект `std::future` для этого результата выполнения асинхронной операции уже был получен в результате предыдущего вызова `get_future()`.

Компонентная функция `std::packaged_task::reset`

Связывает экземпляр `std::packaged_task` с новым результатом выполнения асинхронной операции для той же задачи.

Объявление

```
void reset();
```

Предусловия

У `*this` есть связанная с ним асинхронно выполняемая задача.

Результат

Как при вычислении `*this=packaged_task(std::move(f))`, где `f` — сохраненная задача, связанная с `*this`.

Выдает

Исключение типа `std::bad_alloc`, если под новый результат выполнения асинхронной операции не удастся распределить память.

Компонентная функция `std::packaged_task::valid`

Проверяет у `*this` наличие связанной с ним задачи или результата выполнения асинхронной операции.

Объявление

```
bool valid() const noexcept;
```

Возвращает

true, если у *this есть связанные с ним задача и результат выполнения асинхронной операции, или false в противном случае.

Выдает

Ничего не выдает.

Оператор вызова функции std::packaged_task::operator()

Вызывает задачу, связанную с экземпляром std::packaged_task, и сохраняет возвращенное значение или исключение в связанном результате выполнения асинхронной операции.

Объявление

```
void operator()(ArgTypes... args);
```

Предусловия

У *this имеется связанная с ним задача.

Результат

Вызывает связанную задачу func, как при вычислении INVOKE(func, args...). Если возвращение происходит обычным порядком, сохраняет возвращаемое значение в результате выполнения асинхронной операции, связанном с *this. Если при вызове возвращается исключение, сохраняет его в результате выполнения асинхронной операции, связанном с *this.

Постусловия

Результат выполнения асинхронной операции, связанной с *this, готов и имеет сохраненное значение или исключение. Все потоки, заблокированные в ожидании результата выполнения асинхронной операции, разблокируются.

Выдает

Исключение типа std::future_error с кодом ошибки std::future_errc::promise_already_satisfied, если в результате выполнения асинхронной операции уже есть сохраненное значение или исключение.

Синхронизация

Успешный вызов оператора вызова функции «синхронизируется с» вызовом std::future<ResultType>::get() или std::shared_future<ResultType>::get(), в результате которого извлекается сохраненное значение или исключение.

Компонентная функция std::packaged_task::make_ready_at_thread_exit

Вызывает задачу, связанную с экземпляром std::packaged_task, и сохраняет возвращенное значение или исключение в связанном результате выполнения асинхронной операции, не делая этот результат готовым до выхода из потока.

Объявление

```
void make_ready_at_thread_exit(ArgTypes... args);
```

Предусловия

У **this* имеется связанная с ним задача.

Результат

Вызывает связанную задачу *func*, как при вычислении `INVOKE(func, args...)`. Если возвращение происходит обычным порядком, сохраняет возвращаемое значение в результате выполнения асинхронной операции, связанном с **this*. Если при вызове возвращается исключение, сохраняет его в результате выполнения асинхронной операции, связанном с **this*. Планирует придание готовности связанному асинхронному состоянию при выходе из текущего потока.

Постусловия

В результате выполнения асинхронной операции, связанном с **this*, имеется сохраненное значение или исключение, но готовность ему не придется, пока не начнется выход из текущего потока. Потоки, заблокированные в ожидании результата выполнения асинхронной операции, будут разблокированы при выходе из текущего потока.

Выдает

Исключение типа `std::future_error` с кодом ошибки `std::future_errc::promise_already_satisfied`, если результат выполнения асинхронной операции уже имеет сохраненное значение или исключение. Исключение типа `std::future_error` с кодом ошибки `std::future_errc::no_state`, если у **this* не имеется связанного с ним асинхронного состояния.

Синхронизация

Завершение потока, выполнившего успешный вызов функции `make_ready_at_thread_exit()`, синхронизируется с вызовом функции `std::future<ResultType>::get()` или функции `std::shared_future<ResultType>::get()`, которая извлекает сохраненное значение или исключение.

Г.4.4. Шаблон класса `std::promise`

Шаблон класса `std::promise` предоставляет средства для установки результата выполнения асинхронной операции, который можно получить из другого потока через экземпляр `std::future`.

Параметр шаблона `ResultType` является типом значения, которое можно сохранить в асинхронном результате.

Объект `std::future`, связанный с результатом выполнения асинхронной операции конкретного экземпляра `std::promise`, можно получить путем вызова компонентной функции `get_future()`. Результатом выполнения асинхронной операции становится либо значение типа `ResultType` при вызове компонентной функции `set_value()`, либо исключение при вызове компонентной функции `set_exception()`.

Экземпляры `std::promise` могут быть `MoveConstructible` и `MoveAssignable`, но не `CopyConstructible` и/или `CopyAssignable`.

Определение класса

```
template<typename ResultType>
class promise
{
public:
    promise();
    promise(promise&&) noexcept;
    ~promise();
    promise& operator=(promise&&) noexcept;

    template<typename Allocator>
    promise(std::allocator_arg_t, Allocator const&);

    promise(promise const&) = delete;
    promise& operator=(promise const&) = delete;

    void swap(promise& ) noexcept;

    std::future<ResultType> get_future();

    void set_value(см. описание);
    void set_exception(std::exception_ptr p);
};
```

Конструктор по умолчанию `std::promise`

Создает объект `std::promise`.

Объявление

```
promise();
```

Результат

Создает экземпляр `std::promise` с еще неготовым связанным результатом выполнения асинхронной операции типа `ResultType`.

Выдает

Исключение типа `std::bad_alloc`, если конструктор не в состоянии распределить память под результат выполнения асинхронной операции.

Конструктор-распределитель `std::promise`

Создает объект `std::promise`, используя предоставленный распределитель для выделения памяти под связанный результат выполнения асинхронной операции.

Объявление

```
template<typename Allocator>
promise(std::allocator_arg_t, Allocator const& alloc);
```

Результат

Создает экземпляр `std::promise` с еще неготовым связанным результатом выполнения асинхронной операции типа `ResultType`. Память под результат выполнения асинхронной операции выделяется посредством распределителя `alloc`.

Выдает

Любое исключение, выданное распределителем при попытке выделения памяти под результат выполнения асинхронной операции.

Перемещающий конструктор `std::promise`

Создает один объект `std::promise` из другого, перемещая принадлежность результата выполнения асинхронной операции, связанного с другим объектом `std::promise`, в только что созданный экземпляр.

Объявление

```
promise(promise&& other) noexcept;
```

Результат

Создает новый экземпляр `std::promise`.

Постусловия

Результат выполнения асинхронной операции, связанный с объектом `other` до вызова конструктора, связывается с только что созданным объектом `std::promise`. У объекта `other` больше нет связанного с ним результата выполнения асинхронной операции.

Выдает

Ничего не выдает.

Перемещающий оператор присваивания `std::promise`

Перемещает принадлежность результата выполнения асинхронной операции, связанного с одним объектом `std::promise`, в другой объект.

Объявление

```
promise& operator=(promise&& other) noexcept;
```

Результат

Перемещает принадлежность результата выполнения асинхронной операции, связанного с `other`, в `*this`. Если у `*this` уже есть связанный с ним результат выполнения асинхронной операции, этот результат делается готовым с исключением типа `std::future_error` и кодом ошибки `std::future_errc::broken_promise`.

Постусловия

Результат выполнения асинхронной операции, связанный с объектом `other` до вызова перемещающего оператора присваивания, связывается `*this`. У объекта `other` больше нет связанного с ним результата выполнения асинхронной операции.

Возвращает

`*this`

Выдает

Ничего не выдает.

Компонентная функция `std::promise::swap`

Производит обмен принадлежности результатов выполнения асинхронной операции между двумя объектами `std::promise`.

Объявление

```
void swap(promise& other);
```

Результат

Производит обмен принадлежности результатов выполнения асинхронной операции и задач, связанных с `other` и `*this`.

Поступовия

Результат выполнения асинхронной операции, связанный с `other` до вызова `swap` (если таковой имелся), связан с `*this`. Результат выполнения асинхронной операции, связанный с `*this` до вызова `swap` (если таковой имелся), связан с `other`.

Выдает

Ничего не выдает.

Деструктор `std::promise`

Уничтожает объект `std::promise`.

Объявление

```
~promise();
```

Результат

Уничтожает `*this`. Если у `*this` есть связанный с ним результат выполнения асинхронной операции и этот результат не содержит сохраненного значения или исключения, он становится готовым с исключением `std::future_error`, содержащим код ошибки `std::future_errc::broken_promise`.

Выдает

Ничего не выдает.

Компонентная функция `std::promise::get_future`

Извлекает экземпляр `std::future` для результата выполнения асинхронной операции, связанного с `*this`.

Объявление

```
std::future<ResultType> get_future();
```

Предусловия

У **this* имеется связанный с ним результат выполнения асинхронной операции.

Возвращает

Экземпляр `std::future` для результата выполнения асинхронной операции, связанного с **this*.

Выдает

Исключение типа `std::future_error` с кодом ошибки `std::future_errc::future_already_retrieved`, если объект `std::future` для этого результата выполнения асинхронной операции уже был получен в результате предыдущего вызова `get_future()`.

Компонентная функция `std::promise::set_value`

Сохраняет значение результата выполнения асинхронной операции, связанного с **this*.

Объявление

```
void promise<void>::set_value();
void promise<R&>::set_value(R& r);
void promise<R>::set_value(R const& r);
void promise<R>::set_value(R&& r);
```

Предусловия

У **this* имеется связанный с ним результат выполнения асинхронной операции.

Результат

Сохраняет *r* в результате выполнения асинхронной операции, связанном с **this*, если `ResultType` не относится к типу `void`.

Постусловия

Связанный с **this* результат выполнения асинхронной операции готов и имеет сохраненное значение. Все потоки, заблокированные в ожидании результата выполнения асинхронной операции, разблокируются.

Выдает

Исключение типа `std::future_error` с кодом ошибки `std::future_errc::promise_already_satisfied`, если у результата выполнения асинхронной операции уже есть сохраненное значение или исключение. Любые исключения, выданные копирующим или перемещающим конструктором *r*.

Синхронизация

Несколько конкурентных вызовов `set_value()`, `set_value_at_thread_exit()`, `set_exception()` и `set_exception_at_thread_exit()` выполняются последовательно. Успешный вызов `set_value()` «происходит до» вызова функции `std::future<ResultType>::get()` или вызова функции `std::shared_future<ResultType>::get()`, извлекающей сохраненное значение.

Компонентная функция `std::promise::set_value_at_thread_exit`

Сохраняет значение в результате выполнения асинхронной операции, связанном с `*this`, не делая этот результат готовым до выхода из текущего потока.

Объявление

```
void promise<void>::set_value_at_thread_exit();  
void promise<R&>::set_value_at_thread_exit(R& r);  
void promise<R>::set_value_at_thread_exit(R const& r);  
void promise<R>::set_value_at_thread_exit(R&& r);
```

Предусловия

У `*this` есть связанный с ним результат выполнения асинхронной операции.

Результат

Сохраняет `r` в результате выполнения асинхронной операции, связанном с `*this`, если `ResultType` не относится к типу `void`. Помечает результат выполнения асинхронной операции как имеющий сохраненное значение. Планирует придание готовности связанному результату выполнения асинхронной операции при выходе из текущего потока.

Постусловия

Результат выполнения асинхронной операции, связанный с `*this`, имеет сохраненное значение, но не готов, пока не будет выполнен выход из текущего потока. Потоки, заблокированные в ожидании результата выполнения асинхронной операции, будут разблокированы при выходе из текущего потока.

Выдает

Исключение типа `std::future_error` с кодом ошибки `std::future_errc::promise_already_satisfied`, если результат выполнения асинхронной операции уже имеет сохраненное значение или исключение. Любые исключения, выданные копирующим или перемещающим конструктором `r`.

Синхронизация

Несколько конкурентных вызовов `set_value()`, `set_value_at_thread_exit()`, `set_exception()` и `set_exception_at_thread_exit()` выполняются последовательно. Завершение потока, выполнившего успешный вызов функции `set_value_at_thread_exit()`, «происходит до» вызова функции `std::future<ResultType>::get()` или вызова функции `std::shared_future<ResultType>::get()`, извлекающей сохраненное значение.

Компонентная функция `std::promise::set_exception`

Сохраняет исключение в результате выполнения асинхронной операции, связанном с `*this`.

Объявление

```
void set_exception(std::exception_ptr e);
```

Предусловия

У **this* имеется связанный с ним результат выполнения асинхронной операции. Выражение `(bool)e` вычисляется в `true`.

Результат

Сохраняет *e* в результате выполнения асинхронной операции, связанном с **this*.

Постусловия

Результат выполнения асинхронной операции, связанный с **this*, готов и имеет сохраненное исключение. Все потоки, заблокированные в ожидании результата выполнения асинхронной операции, разблокируются.

Выдает

Исключение типа `std::future_error` с кодом ошибки `std::future_errc::promise_already_satisfied`, если в результате выполнения асинхронной операции уже имеется сохраненное значение или исключение.

Синхронизация

Несколько конкурентных вызовов `set_value()` и `set_exception()` выполняются последовательно. Успешный вызов функции `set_exception()` «происходит до» вызова функции `std::future<Result-Type>::get()` или функции `std::shared_future<ResultType>::get()`, извлекающей сохраненное исключение.

Компонентная функция `std::promise::set_exception_at_thread_exit`

Сохраняет исключение в результате выполнения асинхронной операции, связанном с **this*, не делая этот результат готовым до выхода из текущего потока.

Объявление

```
void set_exception_at_thread_exit(std::exception_ptr e);
```

Предусловия

У **this* имеется связанный с ним результат выполнения асинхронной операции. Выражение `(bool)e` вычисляется в `true`.

Результат

Сохраняет *e* в результате выполнения асинхронной операции, связанном с **this*. Планирует придание готовности связанному результату выполнения асинхронной операции при выходе из текущего потока.

Постусловия

Результат выполнения асинхронной операции, связанной с **this*, имеет сохраненное исключение, но не готов, пока не будет выполнен выход из текущего потока. Потоки, заблокированные в ожидании результата выполнения асинхронной операции, будут разблокированы при выходе из текущего потока.

Выдает

Исключение типа `std::future_error` с кодом ошибки `std::future_errc::promise_already_satisfied`, если в результате выполнения асинхронной операции уже имеется сохраненное значение или исключение.

Синхронизация

Несколько конкурентных вызовов `set_value()`, `set_value_at_thread_exit()`, `set_exception()` и `set_exception_at_thread_exit()` выполняются последовательно. Завершение потока, выполнившего успешный вызов функции `set_exception_at_thread_exit()`, «происходит до» вызова функции `std::future<ResultType>::get()` или вызова функции `std::shared_future<ResultType>::get()`, извлекающей сохраненное исключение.

Г.4.5. Шаблон функции `std::async`

Шаблон функции `std::async` предоставляет простой способ запуска автономных асинхронных задач для задействования доступной аппаратной конкурентности. Вызов функции `std::async` возвращает объект `std::future`, который будет содержать результат выполнения задачи. В зависимости от политики запуска задача либо запускается в асинхронном режиме в своем собственном потоке, либо в синхронном режиме в том потоке, который вызвал в отношении данного фьючерса компонентную функцию `wait()` или `get()`.

Объявление

```
enum class launch
{
    async, deferred
};

template<typename Callable, typename ... Args>
future<result_of<Callable(Args...)>::type>
async(Callable&& func, Args&& ... args);

template<typename Callable, typename ... Args>
future<result_of<Callable(Args...)>::type>
async(launch policy, Callable&& func, Args&& ... args);
```

Предусловия

Выражение `INVOKE(func, args)` является допустимым для предоставленных значений `func` и `args`. Объекты типа `Callable` и все компоненты `Args` являются `MoveConstructible`.

Результат

Создает копии `func` и `args...` во внутреннем хранилище (обозначаемом соответственно `fff` и `xyz...`).

Если `policy` имеет значение `std::launch::async`, запускает `INVOKE(fff, xyz...)` в своем собственном потоке. Возвращенный объект `std::future` станет готовым, как только этот поток завершится, и будет содержать либо возвращаемое значение, либо исключение, выданное в результате вызова функции. Деструктор последнего объекта-фьючерса, связанного с асинхронным состоянием возвращенного `std::future`, блокируется до готовности фьючерса.

Если `policy` имеет значение `std::launch::deferred`, `fff` и `xyz...` сохраняются в возвращенном объекте `std::future` в виде вызова отложенной функции. Первый вызов в отношении фьючерса, совместно использующего то же самое связанное состояние